



Cyprus
University of
Technology

Department of Electrical
Engineering and Computer
Engineering and Informatics

Bachelor Thesis

An Evaluation of a Federated Learning Framework for Detecting Hate Speech and Disinformation on Social Media Platforms

Michael Angelos Demou

Limassol, May 2024

CYPRUS UNIVERSITY OF TECHNOLOGY

Faculty of Engineering and Technology

Department of Electrical Engineering, Computer Engineering, and Informatics

Bachelor Thesis

An Evaluation of a Federated Learning Framework for Detecting Hate Speech and Disinformation on Social Media Platforms

Michael Angelos Demou

Advisor:

Dr Michael Sirivianos

Limassol, May 2024

Copyrights

Copyright © 2024 Michael Angelos Demou

All rights reserved.

The approval of the dissertation by the Department of Electrical Engineering, Computer Engineering, and Informaticsdoes not necessarily imply the approval by the Department of the views of the writer.

Acknowledgements

I would like to especially thank Dr. Michael Sirivianos, Dr. Nikos Salamanos and PhD student Pantelitsa Leonidou of the Department of Electrical Engineering, Computer Engineering and Informatics, for their help and support throughout this thesis. Thank you for giving me the opportunity to undertake a topic of such interest and importance.

ABSTRACT

The spread of harmful content and fake news on social media platforms has become a significant issue, highlighting the need for automatic detection and filtering tools. Machine learning, which enables computers to identify patterns, is frequently employed to develop these tools. However, it involves handling sensitive user data, which must be managed carefully to respect privacy. Federated Learning (FL) addresses this by allowing data to remain on the user's device, rather than being transferred to a central server. In FL, learning occurs locally on individual devices and only model updates are shared with the server, thus minimizing privacy risks. Despite its advantages, FL faces challenges such as statistical heterogeneity, where data is not uniformly distributed across devices. This can result in biased predictions when some classes are over-represented in a client's dataset or when data amounts vary, affecting model influence. This thesis develops a text classifier within a distributed Federated Learning (FL) setup to identify tweets containing harmful content or fake news, focusing on simulating two custom non-IID methods from another study [12] to create real a life scenario for detecting harmful content, as well as a real-life non-IID scenario for disinformation detection. Testing this model on five Twitter datasets representing various types of user misbehavior and misinformation yielded promising results, with F1 scores ranging from 70% to 82%. We further analyzed the model's performance with different numbers of clients and compared it to a centralized approach. Additionally, we experimented with various alpha values affecting the custom non-IID methods to assess the model's performance compared to IID conditions. This comprehensive evaluation confirms the robustness of our FL framework, making it a viable solution for detecting harmful content and disinformation in real-world, non-IID scenarios.

Keywords: Federated Learning, Non-IID, Disinformation, Harmful Content, Text Classification

TABLE OF CONTENTS

ABSTRACT	iv
TABLE OF CONTENTS	v
LIST OF TABLES	vii
LIST OF FIGURES	viii
LIST OF ABBREVIATIONS	ix
1 Introduction	1
1.1 Problem Statement	1
1.2 Aims and Objectives	2
1.3 Structure of the Thesis	2
2 Technical background	3
2.1 Artificial Intelligence	3
2.1.1 Machine Learning	3
2.1.1.1 Overfitting and Solutions	3
2.1.2 Deep Learning	5
2.1.3 Neural Networks	6
2.2 Federated Learning	9
2.2.1 What is Federated Learning	9
2.2.2 How does it work	10
2.2.3 Applications and Use Cases	11
2.3 Natural Language Processing (NLP)	11
2.3.1 Fundamentals of NLP	11
2.3.2 NLP techniques for text analysis	13
3 Research Methodology	14
3.1 Preliminary work	14
3.2 Creating artificial clients for FL	14
3.3 Text Classifier	14
3.4 Datasets	15
3.5 Technologies	16
3.5.1 TensorFlow Federated (TFF)	16

3.5.1.1	What is TFF	16
3.5.1.2	How does it work	17
3.5.1.3	Why TensorFlow Federated was chosen	19
3.5.1.4	Comparison of TensorFlow Federated with other FL frameworks	19
3.5.2	Code Editor	20
3.5.3	Python	20
3.5.4	Libraries Used	20
3.6	Experimental Evaluation	22
3.6.1	Experiments	22
3.6.2	Setup	24
3.6.3	Preparing data for training	24
4	Results/ Discussion	28
5	Future Work	32
	BIBLIOGRAPHY	33

LIST OF TABLES

3.1	Datasets Overview	15
3.2	Framework Comparison	19
4.1	F1 score (standard deviation noted) for hate speech datasets for different alpha variations and IID for 100 clients using quantity skew and distribution-based label imbalance	29
4.2	The performance metrics for the hate speech datasets were evaluated under a non-iid condition using the quantity skew method, as previously discussed, with an alpha=0.5.	29
4.3	The MM-Covid dataset went testing under actual non-iid conditions. This dataset was subjected to experiments conducted over 6 epochs and 60 rounds. The results presented are average values calculated from five repetitions, with the standard deviation noted. This table focuses on the second experiment.	30

LIST OF FIGURES

2.1	Schematic diagram of Dropout [43]	4
2.2	Machine and Deep Learning [26]	6
2.3	Comparison of RNN and Feed-Forward Architectures [37]	8
2.4	Unfolding RNN [37]	9
2.5	Federated Learning Process [29]	10
2.6	Probabilistic Language Model Using Neural Networks [30]	12
3.1	Architecture of TensorFlow Federated [36]	18
3.2	Data distribution across 100 clients for Abusive dataset using Distribution-based Label Imbalance	23
3.3	Data distribution across 100 clients for Abusive dataset using Quantity Skew	23
3.4	Data distribution across 100 clients for MM-Covid dataset	24
3.5	Original Data	25
3.6	Preprocessed Data	25
3.7	Labeled Data	26
3.8	Shuffled Data	26
3.9	Tokenized and Padded Data	27
4.1	Comparison of F1 scores for different alpha variations and different non-iid methods . .	30
4.2	Comparison between AUC and number of clients for MM-Covid dataset over 5 repetitions, 60 rounds and 6 epochs.	31
4.3	Model convergence for Abusive dataset over 5 repetitions for 100 clients. In this graph we show the reason we chose the number of training rounds discussed in Section 3.6.2 . .	31

LIST OF ABBREVIATIONS

Important abbreviations that have been used in the text and need explanation are briefly presented.

FL	Federated Learning
TFF	TensorFlow Federated
GRU	Gated Recurrent Unit
LSTM	Long Short-Term Memory
NLP	Natural Language Processing
AI	Artificial Interlligence
ML	Machine Learning
ReLU	Rectified Linear Unit
CNN	Convolutional Neural Network
RNN	Recurrent Neural Networks
GD	Gradient Descent
BERT	Bidirectional Encoder Representations from Transformers
VSCode	Visual Studio Code
OSN	Online Social Network

1 Introduction

In an era where digital platforms are everywhere, users of Online Social Networks (OSNs) are increasingly exposed to harmful content such as hate speech and misinformation. This flood of content threatens not only social harmony but also the integrity of information. The automated detection of such content is crucial, but it requires sophisticated machine learning (ML) models trained on large and heterogeneous datasets. However, the development and processing of these datasets raises important privacy and security concerns, as they frequently contain sensitive user data. [4]. To solve these difficulties, we propose using a privacy-preserving Federated Learning (FL) system. FL allows for decentralized training of ML models directly on users' devices (the FL "clients"), with just model parameters, not raw data, shared with a central server (the FL "aggregator"). This strategy not only reduces privacy risks by following the strict data protection rules such as the GDPR, but it also uses the most recent, local user data without the need for central collecting. This methodology is ideal for handling dynamic and multimodal data from millions of users across OSNs, providing a strong protection against cyber-attacks and data theft. In this thesis, we look into how federated learning can be used to prevent hate speech and misinformation on platforms such as Twitter. We use TensorFlow Federated (TFF), a powerful framework for building machine learning models on decentralized data. Our model analyzes textual data across many local settings using natural language processing (NLP) approaches and deep learning architectures including Gated Recurrent Units (GRUs) and embedding layers. This decentralized approach protects user privacy and allows the model to benefit from a broad and representative dataset, which is critical for effectively recognizing and categorizing dangerous online content. Our study enhances machine learning for difficult social challenges like hate speech and misinformation, while simultaneously demonstrating a scalable, privacy-centric solution that protects user data integrity and confidentiality. It emphasizes the potential of federated learning to change the way we approach the difficulties posed by dangerous information on digital platforms, with the goal of creating safer and more reliable online environments.

1.1 Problem Statement

Tackling hate speech and misinformation on Twitter, particularly in English-language tweets, is a major challenge for digital security and inclusion. The expansive and fast-paced nature of the platform requires advanced solutions to identify offensive and fake content while maintaining user privacy under non-IID conditions. Traditional methods, based on centralized moderation, fall short in effectively managing the scale and variations of Twitter's global discussion. This thesis introduces a novel approach through FL, using TensorFlow Federated to train machine learning models on decentralized data sources. This method aims to enhance the detection of hate speech in English tweets, as well as detecting misinformation that was spread during COVID-19, by exploiting the diverse information from distributed datasets, all without compromising data privacy. Focusing specifically on Twitter, the research seeks to improve content control practices, ensuring a safer online environment.

1.2 Aims and Objectives

The primary aim of this thesis is to leverage the capabilities of federated learning to enhance the detection and mitigation of hate speech and misinformation across online platforms. In the face of growing concerns about privacy and non-IIDness, this research seeks to demonstrate how a decentralized machine learning model can process sensitive textual data across multiple nodes—each holding local data—without compromising user privacy under non-IID conditions. By adopting TensorFlow Federated (TFF) for model development and employing advanced Natural Language Processing (NLP) techniques alongside deep learning architectures, the thesis aims to create a robust, privacy-preserving model capable of accurately classifying instances of hate speech and misinformation. Objectives include assessing the effectiveness of federated learning in dealing with diverse and representative datasets, thereby improving the model’s ability to identify hate and fake content across various demographics and linguistic expressions. Additionally, the research intends to explore the implications of federated learning for non-IIDness, providing insights into its scalability and applicability to broader social issues. Ultimately, this thesis attempts to contribute to safer online environments by offering a scalable, privacy-focused solution to the common challenge of online hostility and fake news.

1.3 Structure of the Thesis

This thesis follows the structure below: After the introduction, the problem statement and the aims of the thesis, the technical background is mentioned. At this point, basic definitions are presented that are important for understanding the thesis. Then the research methodology is presented, i.e. the process that was done to create the system and the tools that have been used. In the fourth chapter, the system that has been created is demonstrated by showing the results. Finally, the fifth chapter mentions the possible future work of this study.

2 Technical background

This thesis requires an understanding of machine learning principles and algorithms, as well as natural language processing techniques. It introduces basic concepts in artificial intelligence, with a focus on FL, to lay the foundation for the study. It also discusses natural language processing that was used further for the creating of the model. Challenges such as data privacy, model convergence in Federated settings, and the complexity of social media language processing are also discussed, emphasizing the innovative approach to maintaining user privacy while effectively monitoring content.

2.1 Artificial Intelligence

Artificial Intelligence has gained a lot of popularity in our time, consisting of the development of machines that mimic human intelligence and have the ability to learn and perform tasks like humans. These machines can be improved through experience and can perform various human functions. In essence, artificial intelligence refers to systems that react to stimuli in a way similar to human response, incorporating the capacity for thought, judgment, and intention [22]. Artificial intelligence is based on algorithms that enable automated decision making without the need for specific instructions, often in collaboration with technologies such as machine learning and data analytics. This combination helps in more sophisticated and intelligent analysis and decision making. The continuous evolution of artificial intelligence has the potential to significantly improve the quality of our lives in many areas.

2.1.1 Machine Learning

In machine learning, data is analysed to discover hidden patterns. When something important is found, software designers can use this information to solve practical problems through data analysis. Machine learning techniques are divided into two main categories: supervised and unsupervised learning. [23] Supervised learning allows systems to apply prior knowledge to new data, using labelled examples to make predictions. After being trained on a particular data set, they build a model that can predict outcomes for new inputs, improving its accuracy by comparing predicted and actual outcomes. [23] On the other hand, unsupervised learning works with unlabeled data, trying to find structures or patterns within the data. This approach focuses on exploring the data to understand the underlying structures without specific instructions on what it is looking for. [23] More generally, machine learning teaches computers to make inferences and decisions based on their past experiences, identifying patterns and analysing past data to reach logical conclusions. This process enables automated data analysis, saving time and improving efficiency in business processes. [24]

2.1.1.1 Overfitting and Solutions

Overfitting is a critical concern in machine learning, compromising the generalizability of predictive models. Overfitting happens when a model learns the training data so thoroughly that it captures noise and random fluctuations rather than the true data distribution. This results in poor performance on unknown data since the model's predictions are tuned to the unique characteristics of the training set rather than

broad trends. Overfitting can be mathematically defined in terms of model complexity; a model with too many parameters in relation to the amount of observations may display high variance, fitting the training data closely but resulting in a loss of predicted accuracy on fresh data [41]. The bias-variance trade-off formalizes this concept by defining the balance between error due to bias (error caused by incorrect assumptions in the learning algorithm) and error due to variance (error caused by sensitivity to tiny variations in the training set).

Dropout, provides a compelling solution to the overfitting problem [41]. The dropout strategy thins a neural network by randomly "dropping out" a subset of units (neurons) after each training iteration. This procedure can be expressed mathematically as a kind of Bernoulli sampling for each neuron. Each neuron is kept with probability p and dropped with probability $1-p$. Applying dropout to a layer with output y results in a thinned output $r \odot y$, where $\odot$ represents element-wise multiplication and r is a vector of independent Bernoulli random variables with a probability of 1. At test time, no dropout is used, but the network weights are scaled by p , a technique known as weight scaling, to imitate the impact of averaging the predictions of the exponentially numerous thinner networks.

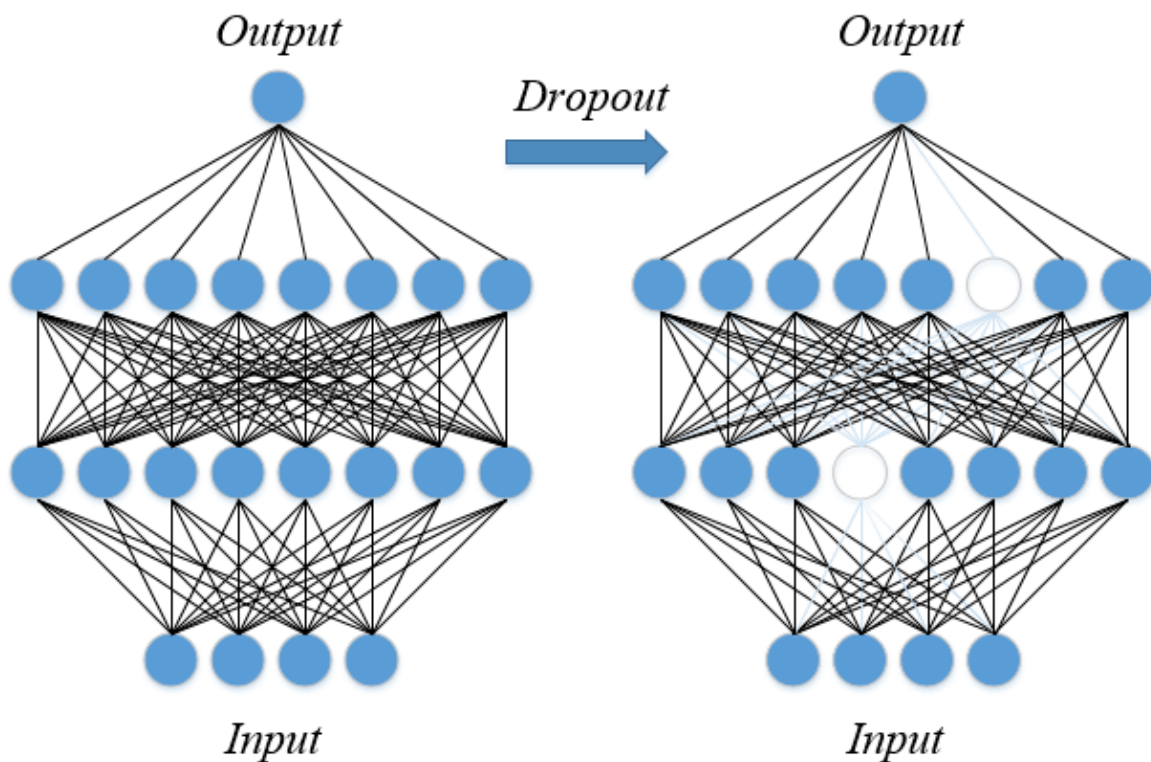


Figure 2.1: Schematic diagram of Dropout [43]

Aside from dropout, various regularization strategies play an important role in preventing overfitting. L1 regularization (lasso) and L2 regularization (ridge) are two popular approaches that add a penalty term to the loss function used to train the model. L1 regularization adds a term $\lambda \sum_i |w_i|$, where w_i represents the model's weights and λ controls the strength of the regularization. This strategy promotes sparsity in the model's parameters, effectively performing feature selection by setting specific weights to zero. In contrast, L2 regularization incorporates a penalty term $\lambda \sum_i w_i^2$, which discourages big weights by penalizing them proportionally to the square of the weights. Both L1 and L2 regularization change the optimization target, forcing the model to balance the fit to the data against the complexity of the model representation, resulting in models that generalize better to new data [42].

The underlying goal of machine learning is to construct models that generalize well to previously unseen data, which emphasizes the critical relevance of overfitting prevention measures such as dropout and regularization. These tactics ensure that models don't just memorize the training data, but also catch the underlying patterns that allow for accurate predictions across multiple datasets. As the complexity of machine learning models and the variety of tasks they are used for grows, employing regularization approaches to improve generalizability becomes increasingly important. Finally, the ability to reduce overfitting is critical in harnessing the predictive power of machine learning models, which drives breakthroughs and applications across a wide range of areas.

2.1.2 Deep Learning

Deep learning is a method of machine learning that allows computers to analyse data at different levels in order to make predictions or understand complex information. This technology has applications in many fields, such as in automated vehicles, where it helps cars recognise traffic signs or distinguish people from objects, and in voice control for devices such as mobile phones, tablets, TVs and speakers. Through deep learning, computers can learn to recognise and interpret images, text or sounds, reaching levels of accuracy that sometimes exceed those of humans. The process of training these models is done using large data sets and complex structures, such as neural networks, which have multiple layers.

Deep learning is a very powerful method. However, as with any effective technology, there are challenges that need to be addressed. Deep learning systems can exhibit biases that come from the data with which they have been trained. This means that biases present in the training data can be carried over into the decisions of the deep learning system. There have been cases where deep learning systems have shown discrimination against certain groups of people. For example, the company Amazon was forced to stop using a recruitment tool based on artificial intelligence because it was shown to have a bias against women [25].

When it falls into the wrong hands, advanced deep learning technology has the potential to be used for negative purposes. Deep learning, by developing the ability to produce realistic images and sounds as well as to process and produce natural language, carries the risk of being used to manufacture illegal material with the help of artificial intelligence. A major incident occurred in 2018 with the release of FakeApp, an app that allowed changing faces in videos using this technology. This led to the creation of fake pornographic videos with the faces of celebrities and politicians, causing great concern [25].

Another risk associated with AI learning is the opposite reactions it can have. The algorithms of this technology, because of the way they are designed, may work in ways we don't expect or seem strange

to us. Adding to the complexity, the neural network process is such that it does not easily allow for the detection and correction of all possible errors that could exist [25].

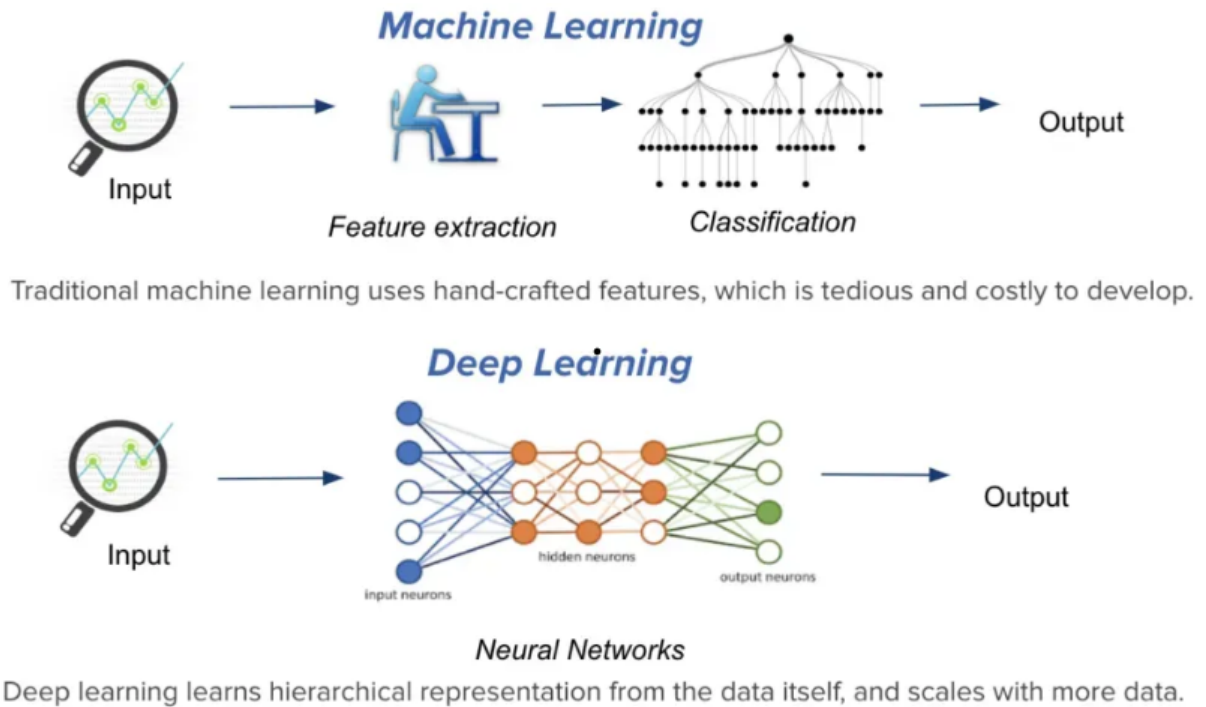


Figure 2.2: Machine and Deep Learning [26]

2.1.3 Neural Networks

Deep learning algorithms are built on neural networks, which are modeled after biological neural networks found in animal brains. They are designed to mimic how biological neurons communicate, with input and output layers as well as one or more hidden layers. These hidden layers convert the input into something that the output layer can use, making it easier to learn nonlinear relationships.

Neural networks have a few basic components: neurons, weights, biases, and activation functions. Each neuron in the network gets input data, which is subsequently weighted—a process that determines how the input influences the output. A bias is applied to this weighted input, allowing the neuron to change its activity level. This sum is then passed through an activation function, which computes the neuron's output based on the input received.

Weights and biases are critical components of neural networks. Weights change the strength of the input signal, whereas biases vary the activation function to improve data fitting. Initially set to random values, these parameters are fine-tuned during the training process to reduce the difference between the network's predictions and the actual data results.

Activation functions introduce non-linearity into neural networks, allowing them to learn complicated patterns. Without these functions, a neural network would only conduct linear transformations, unable to comprehend the complex relationships in the data.

The Sigmoid Activation Function is one such function that returns values ranging from 0 to 1. This feature makes it perfect for models that forecast probability. However, its use has decreased due to restrictions

such as the vanishing gradient problem, which impedes the learning process in deep networks.

On the other hand, the ReLU (Rectified Linear Unit) function has gained popularity because of its efficiency, particularly in deep neural networks. ReLU activates a neuron only when the input is positive, else it outputs zero. This simplicity helps to avoid the vanishing gradient problem, resulting in faster and more effective learning.

In a neural network, the forward pass includes passing input data through each neuron—applying weights, adding biases, and activating the sum—across the hidden layers until it reaches the output layer. The final activation generates the network's prediction. The network then computes the loss, which assesses the prediction's accuracy. Backpropagation allows the network to change its weights and biases to reduce loss while also refining its predictions.

Within the diverse architecture of neural networks, Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) are distinguished by their specialized capabilities. CNNs excel at processing data with a grid-like topology, such as images, making them the foundation for computer vision tasks. RNNs, on the other hand, are designed to deal with sequential data, making them ideal for natural language processing and time series analysis tasks.

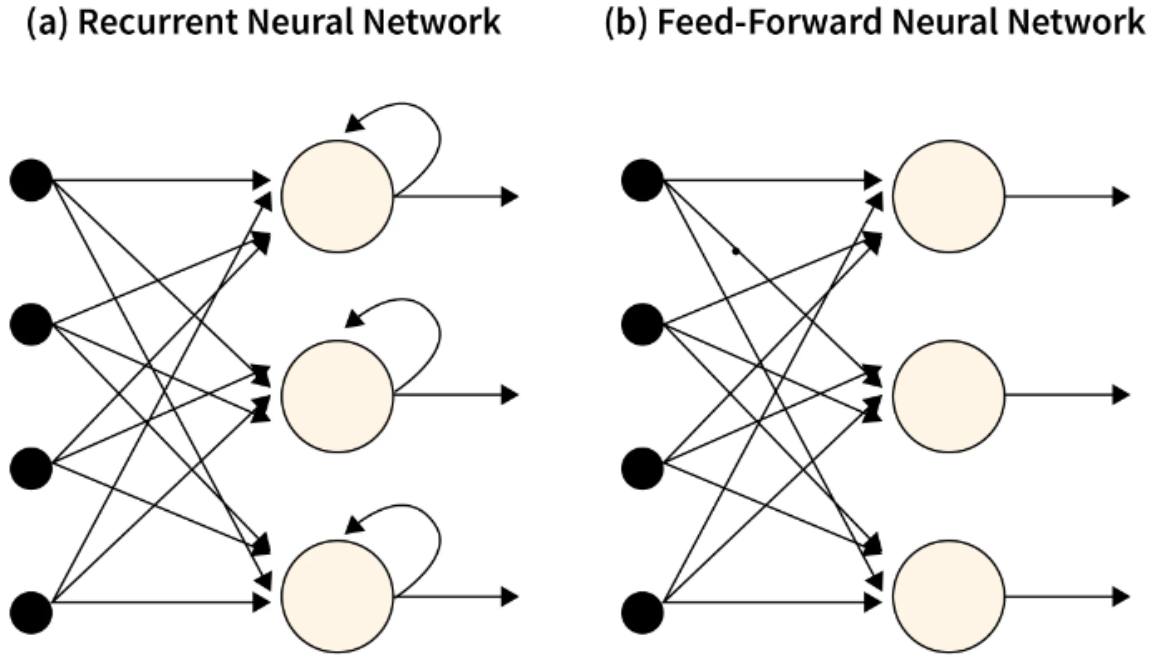


Figure 2.3: Comparison of RNN and Feed-Forward Architectures [37]

RNNs are defined by their memory, which allows them to influence the output of a layer based on previous computations, thereby adding a temporal dimension. This capability is critical for processing data sequences, which enables applications such as language modeling, translation, speech recognition, and even video analysis.

RNNs are mathematically based on a simple yet powerful formula that recursively processes each element of a sequence while maintaining a hidden state that encapsulates information learned from previous elements. For the hidden state update at time t , this is expressed as $h^{(t)} = f(\mathbf{W} \cdot \mathbf{h}^{(t-1)} + \mathbf{U} \cdot \mathbf{x}^{(t)} + \mathbf{b}_h)$, For the output at time t , this is expressed as $o^{(t)} = g(\mathbf{V} \cdot \mathbf{h}^{(t)} + \mathbf{b}_o)$, where $\mathbf{x}^{(t)}$ is the input vector at time step t , $\mathbf{h}^{(t)}$ is the hidden state at time step t , which serves as the "memory" of the network, incorporating information from the current input and past hidden states, $\mathbf{o}^{(t)}$ is the output vector at time step t , $\mathbf{W}$, $\mathbf{U}$, and $\mathbf{V}$ are the weight matrices for the hidden-to-hidden, input-to-hidden, and hidden-to-output connections, respectively, $\mathbf{b}_h$ and $\mathbf{b}_o$ are the bias vectors for the hidden layer and output layer, respectively, and f and g are the non-linear activation functions [38].

Despite their strengths, RNNs face challenges, most notably the vanishing gradient problem, which complicates learning dependencies from long sequences. Gated Recurrent Units (GRUs) were introduced as a solution to improve the network's ability to detect long-term dependencies. GRUs simplify the RNN architecture without significantly reducing performance, making them useful for a variety of tasks [33].

GRUs use gating mechanisms to regulate the flow of information. They decide what information is necessary to maintain or discard, so resolving the vanishing gradient issue. The GRU design contains an update gate which defines how much of the previous memory to keep and a reset gate, which determines how to combine the new input with the previous memory. These mechanisms enable GRUs to learn from data points distributed over extended sequences, which is critical for tasks that involve comprehending the context and nuances in linguistic data [39].

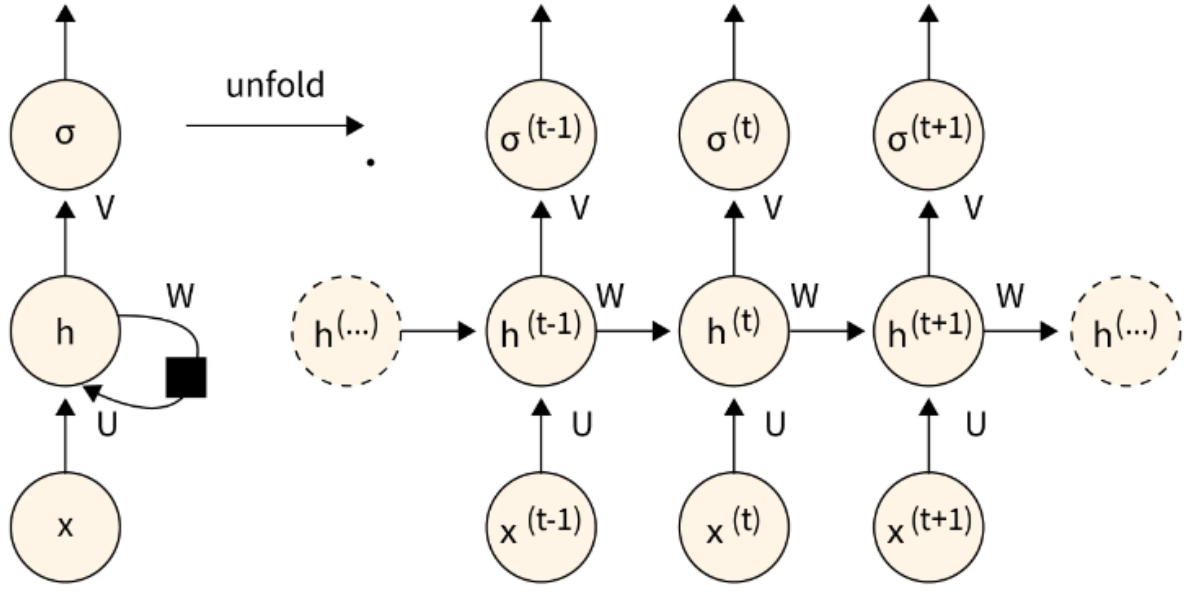


Figure 2.4: Unfolding RNN [37]

2.2 Federated Learning

2.2.1 What is Federated Learning

Federated Learning (FL) represents a major change in the field of machine learning (ML) and artificial intelligence (AI), emphasizing privacy and efficiency in training models on decentralized data sources. Unlike traditional ML approaches that rely on aggregating massive datasets for training, FL allows models to learn from data distributed across multiple devices or servers while keeping the data localized [4]. This approach not only mitigates privacy concerns by avoiding the transfer of raw data over the network, but also leverages the computational power of devices to reduce the burden on central servers [16].

At its core, FL involves distributing a global model to devices (clients) that each hold a subset of the global data. These devices independently compute updates to the model based on their local data. The updates are then sent to a central server, where they are aggregated to improve the overall model, which is then redistributed to clients for further rounds of training [4]. This process continues iteratively, improving the performance of the model without direct access to the underlying data, thus maintaining user privacy.

Despite its promising advantages, FL introduces several technical challenges that need to be addressed to fully exploit its potential. One of the primary issues is the handling of non-IID data across different clients. In many real-world scenarios, the data held by each client can vary significantly in terms of its distribution, making it difficult to ensure that the global model performs well across clients [27]. Moreover, the iterative process of updating the global model requires efficient communication protocols to minimize bandwidth usage and ensure timely updates, especially in environments with limited connectivity [13].

In addition, the decentralized nature of FL raises concerns about the security and integrity of model updates. Malicious clients could introduce compromised updates to affect the global model, necessitating the existence of robust algorithms and security measures to detect and mitigate such threats [28]. Ensuring

the convergence of the global model under these conditions is another critical challenge, as traditional optimization techniques may not be directly applicable in the Federated environment. Addressing these challenges is crucial to advance FL from a theoretical framework to practical applications.

2.2.2 How does it work

The general FL process involves multiple rounds (R) and N number of clients. Each of these clients holds its own local data samples, which are used to train local models. This local model training is a key feature of FL, allowing each participant to contribute to the learning process without exposing its data. In round 0, the server generates a global model with random parameters, which are sent to each client. Then, in each round R , K of the N participants are randomly selected; each participant i locally computes the training gradients according to its local dataset D_i and sends the updated parameters to the server. The central server aggregates these updates to improve a global model. A common method for this aggregation is the FedAvg algorithm, where the server calculates an average of the updates. The latter computes the global parameters $\theta_R = \sum_{i=1}^K \frac{\theta_i}{K}$ and sends them to all N participants for the next round. After a certain number of rounds (R), the model is finalized with the θ_R parameters. The algorithms used for training can vary, but the principle remains that the raw data remains on the device, ensuring privacy [16].

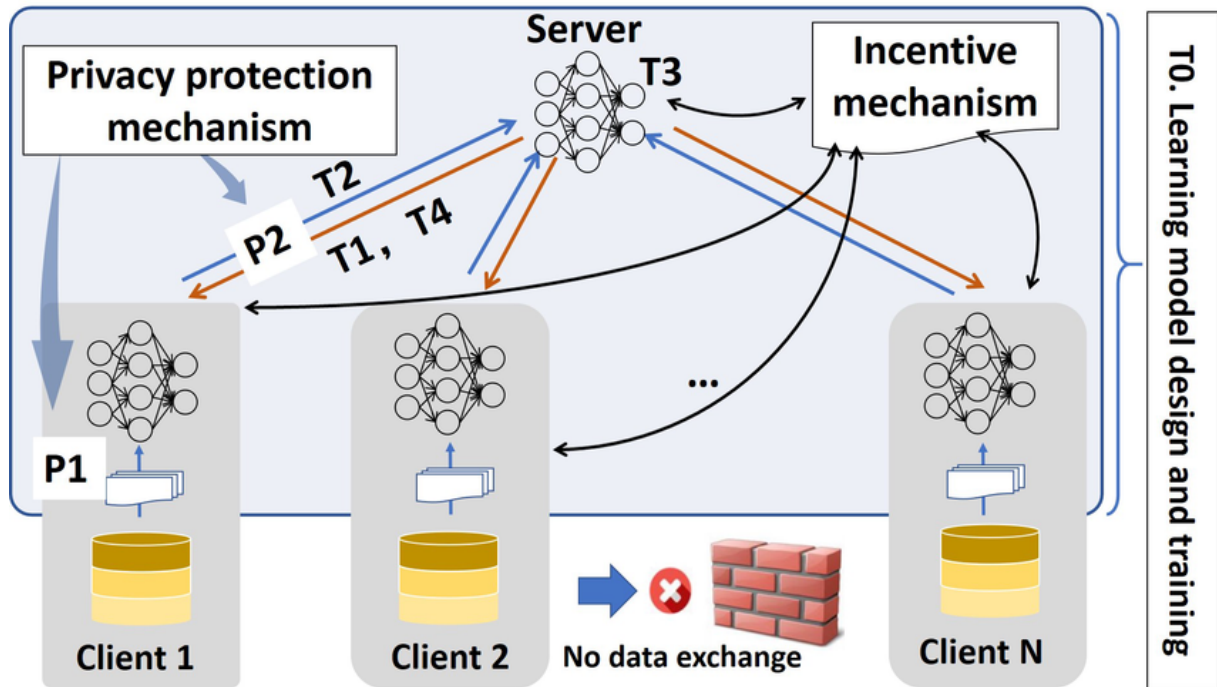


Figure 2.5: Federated Learning Process [29]

Privacy is further enhanced in FL through techniques such as differential privacy, which introduces noise in model updates, and through SMPC or homomorphic encryption (HE), which techniques encrypt updates. These methods ensure that the aggregation process does not compromise the privacy of individual data points [2].

2.2.3 Applications and Use Cases

FL is flexible and can be applied to a wide range of machine learning models, as long as they can be adapted to incorporate updates or partial derivatives necessary to optimize the model parameters. This feature makes FL compatible with numerous algorithms, especially those based on gradient descent (GD) - an optimization technique that is central to many machine learning processes. Models such as linear and logistic regression, alongside neural networks, are commonly optimized using GD, which demonstrates the applicability of FL to various machine learning methodologies, including linear support vector machines and matrix factorization-based methods such as collaborative filtering [25].

Beyond the field of supervised learning, FL extends its usefulness to unsupervised learning scenarios. For example, the k-means clustering algorithm aligns well with the distributed nature of FL, where updates facilitate centroid adaptations. Similarly, principal component analysis, traditionally not associated with update-based methods, can be adapted to FL through distributed computation strategies, such as the power iteration method, allowing eigenvector and eigenvalue computations on decentralized datasets.

However, some algorithms, such as $k - NN$, which requires data point memorization, prove incompatible with FL due to its inherent data privacy protocols. The effectiveness and feasibility of FL also depends on the nature of the available data, particularly favoring scenarios where data privacy concerns preclude centralized data storage or processing.

An illustrative application of FL's capabilities can be seen in the field of natural language processing (NLP), specifically in enhancing virtual keyboard functions such as next word prediction. Recurrent neural networks, commonly used for this purpose, benefit significantly from FL, as they learn directly from user-generated text data on devices without compromising data privacy. This approach not only ensures that models are trained in authentic, everyday language usage, which differs from the formal language found in traditional text corpora such as Wikipedia, but also aligns with privacy-preserving principles. Through FL, as users interact with their devices, they flag unintended data, which is immediately used to improve the model, ensuring that updates are calculated in real-time and discarded after use to meet privacy standards [25].

2.3 Natural Language Processing (NLP)

2.3.1 Fundamentals of NLP

Natural Language Processing (NLP) is a field that lies at the intersection of computer science, artificial intelligence (AI) and linguistics and aims to make machines capable of understanding, interpreting and producing human languages [30]. The ambition of NLP is not just to enable interaction between humans and machines, but to enable machines to understand language as humans do - capturing nuances, emotions, and even cultural specificities. NLP incorporates insights from linguistics to understand the structure and meaning of language, from computer science to process data and create models, and from artificial intelligence to apply these models to tasks such as translation, sentiment analysis, and more.

NLP includes several key elements: syntax (the structure of sentences), semantics (the meaning of words and sentences), and pragmatics (the use of language in context) [30]. Each area presents unique challenges, such as analyzing complex grammatical structures, resolving ambiguity in word meanings, and

understanding meanings that arise from context. It makes extensive use of probabilistic models to make predictions about language.

For example, the probability of a word appearing in a sentence or the probability of a particular sequence of words can be modelled using statistical methods. A well-known example is the n-gram model, where the probability of a word depends on the occurrence of the previous $N - 1$ words and is calculated as $P(W_n|W_{n-1}, \dots, W_{n-(n-1)})$ where w is denoted as the current word in the sequence.

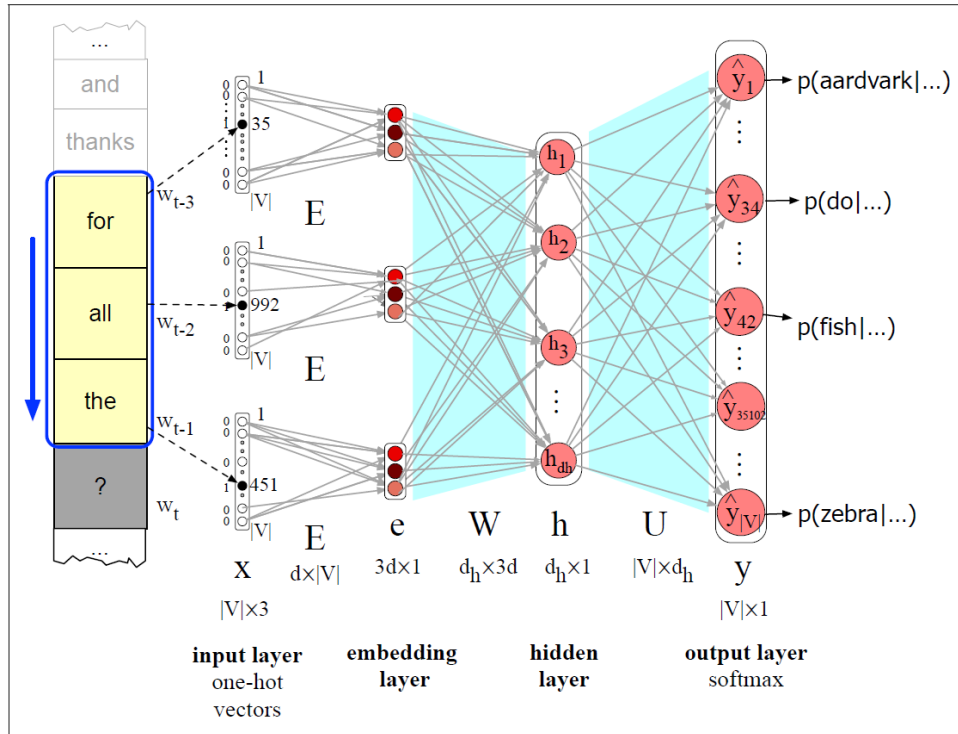


Figure 2.6: Probabilistic Language Model Using Neural Networks [30]

Moreover, idiomatic expressions and the variability of human language pose additional obstacles for NLP systems, requiring sophisticated models for accurate language interpretation and generation.

The digital age, marked by the rise of the Internet and social media, has exponentially increased the availability of text data, revolutionizing NLP. This surge in data has been instrumental in the transition from rule-based to data-based approaches, enabling the development of models that learn language patterns directly from large datasets [31]. The availability of diverse and extensive datasets is crucial for training efficient NLP models capable of understanding human language variations.

2.3.2 NLP techniques for text analysis

Text pre-processing is a fundamental step in NLP that prepares the raw text for further analysis and model training. This process includes several basic steps: tokenization (parsing the text into individual words or tokens), stemming (reducing words to their root form), lemmatization (converting words to their basic or lexicographic form) and removing stopwords (elimination of common words that add little semantic value to the analysis) [32]. These steps are crucial to reduce complexity and focus on the meaningful components of the text. Regular expressions play a key role in text cleaning, helping to remove irrelevant characters, break up sentences, and identify patterns within the text, thereby improving the pre-processing phase [30].

Feature extraction transforms text into a form that machine learning algorithms can understand by extracting meaningful features from the raw data. The bag-of-words model represents text as a collection of independent words, ignoring order but preserving frequency. TF-IDF (Term Frequency-Inverse Document Frequency) improves on this model by weighting terms based on their frequency in documents, highlighting the importance of certain words in a specific context. Word embeddings such as Word2Vec and GloVe offer a more nuanced approach, representing words in continuous vector spaces, capturing semantic similarities. Sequence models, such as LSTM (Long Short-Term Memory) and GRU (Gated Recurrent Units), have revolutionized text analysis by understanding sequences, with GRU providing a more efficient alternative to LSTM, without to compromise the ability to capture long-term dependencies in the text [33]. BERT (Bidirectional Encoder Representations from Transformers) further advances the field by capturing deep contextual meanings from text [34].

NLP includes a wide range of tasks: sentiment analysis assesses the sentiment expressed in text - nominal entity recognition (NER) identifies and categorizes key text entities into predefined categories - part-of-speech tagging assigns parts of speech to each word - and machine translation converts text from one language to another. These tasks are tackled using various models and algorithms, including decision trees, support vector machines for classification tasks, neural networks for learning complex patterns, and transformers for handling sequence-based tasks with remarkable efficiency.

Evaluation of NLP models includes metrics that measure their performance and accuracy. Common metrics include accuracy (the percentage of correct predictions), precision (the ratio of true positive predictions to all positive predictions), recall (the ratio of true positive predictions to all true positive predictions), and F1 score (an average of precision and recall), which metrics will be analyzed next. For translation work, the BLEU (Bilingual Evaluation Understudy) score is widely used, which measures the quality of the machine-translated text against a set of reference translations [35].

3 Research Methodology

For the present thesis, which concerns the development of a model, a specific methodology was used in order to increase efficiency and organizational functionality. The following sections detail the application of this methodology.

3.1 Preliminary work

As mentioned above, this thesis focuses on the application of FL in the field of Natural Language Processing (NLP). In order to understand and perform such a task, several concepts were explored, such as Machine Learning and Deep Learning (see Section 2.1), which provide the statistical and computational background for manipulating complex linguistic data, as well as Federated Learning (see Section 2.2). Further, various concepts of NLP were discussed (see Section 2.3), delving into how machines process and produce human-like text.

3.2 Creating artificial clients for FL

In our federated learning (FL) simulation, "clients" are represented as an artificial construct within a centralized computational framework rather than as genuine physical devices or independent entities. This artificial data segmentation into clients is critical for assessing FL algorithms under a variety of controlled conditions that simulate real-world situations. To mimic both IID and non-IID data distributions among clients, we start with a centralized dataset that is methodically partitioned using predefined algorithms. For IID scenarios, the dataset is shuffled at random and delivered uniformly to all clients, ensuring that each client obtains a representative sample of the overall data distribution. In non-IID settings, we use methods such as distributed-based label imbalance and quantity skew [12]. Distributed-based label imbalance is done by ensuring that each party is allocated a proportion of the samples of each label according to Dirichlet distribution, whilst quantity skew modifies the volume of data each client retains using similar probabilistic approaches. These partitioning schemes are intended to represent the diverse and uneven data conditions that are common in decentralized real-world contexts.

3.3 Text Classifier

The input of the classifier is the text of the tweets. We used TensorFlow Keras for the implementation of the classifier. The sequential ML pipeline starts with an embedding layer, bidirectional GRU units with regularization and a dropout of $p=0.5$. The output layer is a sigmoid activation dense layer for binary classification. TFF framework offers a function that wraps a Keras model for its use in the federated training simulation. Utilizing federated learning, the model trains across multiple devices, reducing computational load and adapting to diverse data distributions with strategies like distributed-based label imbalance and quantity skew [12]. This approach allows for efficient and scalable classification of offensive and misinformation content.

3.4 Datasets

Based on previous research on Twitter misbehavior and misinformation, we chose the datasets listed below for experimental examination. To simplify the FL work for users, we binarize the classification problem by merging many harmful classifications into a single "offensive" class across all hate speech datasets and "fake" for the misinformation dataset. We list both the original and final binary classes below.

Table 3.1: Datasets Overview

Dataset	Number of Tweets	Original		After Binarization	
		Classes	Percentage	Classes	Percentage
Abusive Dataset	~86K	Hate Abusive Normal	5.8% 31.6% 62.6%	Offensive Neither	37.4% 62.6%
Offensive Dataset	~25K	Hate Offensive Normal	6% 77% 17%	Offensive Neither	83% 17%
Hateful Dataset	~16K	Racism Sexism Normal	12% 20% 68%	Offensive Neither	32% 68%
Sarcastic Dataset	~61K	Sarcastic Normal	10.5% 89.5%	Offensive Neither	10.5% 89.5%
MM-Covid Dataset	~10K	Fake Real	32% 68%	Fake Real	32% 68%

Abusive Dataset [6] consists of ~100K tweets each consisting of two columns: "tweet" and "label". The "tweet" column contains text data representing individual tweets, extracted from Twitter. The "label" column is categorized into four classes, Abusive, Hate, Spam and Normal. Because there are more sophisticated techniques to handle spam profiles we removed the spam label [7]. The labels now are as follow: "Abusive" (31.6%), "Hate" (5.8%), and "Normal" (62.6%). Final binary classes: 37.4% "Offensive" and 62.6% "Neither".

Offensive Dataset [8] consists of ~25K tweets each consisting of two columns: "tweet" and "label". The "tweet" column contains text data representing individual tweets, extracted from Twitter. The "label" column is categorized into three classes: "Hate" (6%), "Offensive" (77%), and "Normal" (17%). The "label" column categorizes each tweet into one of two categories, indicating the nature of the content ("offensive", "neither"). Final binary classes: 83% "Offensive" and 17% "Neither".

Hateful Dataset [9] consists of ~16K tweets each consisting of two columns: "tweet" and "label". The "tweet" column contains text data representing individual tweets, extracted from Twitter. The "label" column is categorized into three classes: "Racism" (12%), "Sexism" (20%), and "Normal" (68%). Final binary classes: 32% "Offensive" and 68% "Neither".

Sarcastic Dataset [10] consists of $\sim 61\text{K}$ tweets each consisting of two columns: "tweet" and "label". The "tweet" column contains text data representing individual tweets, extracted from Twitter. The "label" column is categorized into two classes: "Sarcastic" (10.5%) and "None" (89.5%). Final binary classes: 10.5% "Offensive" and 89.5% "Neither".

MM-Covid Dataset. The original dataset is presented in [11]. In this thesis, we experimented with a part of the hypergraph structure for the MM-COVID presented in [45]. Specifically, we consider each hyperedge (set of cascades) as one client which contains only the tweets of the original cascades.

The data consists of $\sim 10\text{K}$ claims or statements related to news, along with metadata and fact-checking references each consisting of nine columns: "news id", "agency", "claim", "fact url", "label", "lang", "ref source", "ref source url", and "statement". The 'news_id' column, is a unique identifier for each news item. The "agency" column, denotes the organization that is either making the claim or reporting the news. The "claim" column, is a short description or headline of the news item. The 'fact_url' column, is a link to a webpage providing fact-checking details or further information about the claim. The "lang" column, is the language of the claim. The 'ref_source' column, contains the reference source of the claim or the news item, the original publication. The 'ref_url' column, is the URL to the original publication and the "statement" column contains the actual text of the claim, news item, or the statement in question. Final binary classes: 32% "Fake" and 68% "Real".

3.5 Technologies

3.5.1 TensorFlow Federated (TFF)

3.5.1.1 What is TFF

TensorFlow Federated (TFF) is an open-source framework developed by Google for machine learning and other computations on decentralized data. TFF is designed to facilitate the development of federated learning algorithms.

The TFF incorporates the principles of FL, ensuring that ownership and control of data is retained by data creators, thus aligning with contemporary concerns around data privacy regulations such as the General Data Protection Regulation (GDPR). Furthermore, TFF provides researchers and developers with the tools to simulate an FL environment, experiment with new algorithms, and prototype their solutions while addressing the challenges inherent in distributed computing.

The framework supports a set of APIs that allow implementing custom FL algorithms and experimenting with advanced machine learning techniques. Leveraging TensorFlow's powerful environment, TFF enables the efficient and scalable execution of federated computations, opening new avenues for collaborative and privacy-preserving machine learning applications in diverse fields such as healthcare, finance, and telecommunications.

3.5.1.2 How does it work

Strategy Layer

At the highest level, TFF defines the strategies for FL, such as FedAvg and FedSGD. These strategies regulate how local model updates from various clients are aggregated to update the overall model.

- FedAvg is an algorithm where local models are trained on client devices and their parameters are averaged to update the overall model. It is calculated according to the formula below [4].

$$W = \frac{1}{N} \sum_{k=1}^K n_k W_k$$

K : Total number of clients

W : Global model

W_k : The local update on client k

N : The total number of data points across all clients

n_k : The number of data points

- FedSGD is a simpler approach where the slopes of a global model are calculated for each customer and then averaged. It is calculated according to the formula below [4].

$$W_{t+1} = W_t - \eta \left(\frac{1}{K} \sum_{k=1}^K g_k \right)$$

K : Total number of clients

W_{t+1} : The parameters of the global model after the update

w_t : The current parameters of the global model

η : The learning rate

g_k : The computable gradient from client k based on its local data

- Sum and Mean refer to the aggregation functions that can be applied to updates received from clients.
- DPQueries and DP represent Differential Privacy strategies that can be applied to ensure that the aggregation process preserves user privacy by adding noise to the aggregated data or queries [36].

Security Layer

This layer includes the security protocols used in TFF, such as the application of Differential Privacy (DP) to ensure the privacy of customer data during the FL process [36].

Model Layer

This layer defines the types of models that can be trained using TFF.

- NN/RNN/CNN: Neural Networks, Recurrent Neural Networks and Convolutional Neural Networks, which can be constructed and trained.
- Keras/TensorFlow: TFF is compatible with models built using the Keras API or the TensorFlow framework, allowing developers to use familiar tools to build the learning models [36].

Run-time Layer

This layer contains the execution environment of the FL process.

- Executor: This is the TFF component responsible for performing the computations, which can be

the actual training or pooling steps.

- **Client:** Represents the client devices or nodes participating in learning. Each client works on local data to compute updates to the model.
- **gRPC/proto:** These terms refer to the communication protocols used to interact between clients and the executor. gRPC is a high-performance, open-source global RPC framework, while proto refers to protocol isolators, a language- and platform-neutral, extensible mechanism for serializing structured data [36].

When an FL job starts, each participating client retrieves the current global model from the executor and performs local training on its own dataset. The parameters or gradients of the local models are then sent back to the server using gRPC/proto communication. The server applies the specified aggregation strategy (eg FedAvg or FedSGD) to update the global model. If DP is implemented, the aggregation step will also include mechanisms to ensure data privacy is maintained. The updated global model is then distributed back to the clients for the next round of training, and this process is repeated until the model converges or meets the desired performance criteria.

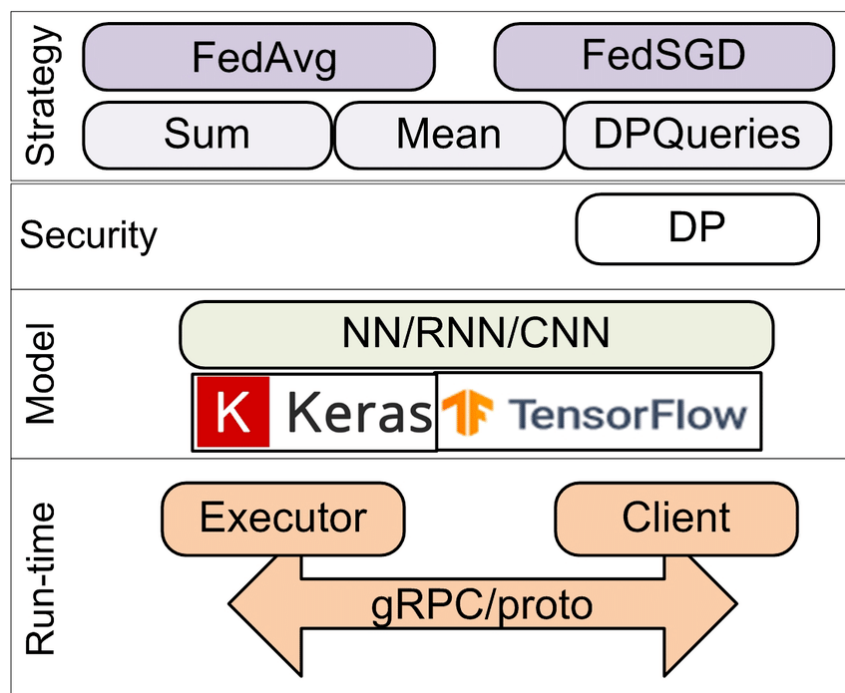


Figure 3.1: Architecture of TensorFlow Federated [36]

3.5.1.3 Why TensorFlow Federated was chosen

TensorFlow Federated (TFF) has been chosen as the main framework for developing the adversarial speech problem model, which works across devices using Federated Learning concepts. The decision to use TFF stems from its integration with TensorFlow, a platform that is already well-known, ensuring a smooth transition and more efficient model development. TFF’s ability to handle decentralized data is well suited to the project’s requirement to process data distributed across multiple devices. This perfectly aligns with the FL model, where data privacy and local computing are of great importance. Additionally, TFF’s two-tier API architecture allows us to leverage high-level interfaces to implement Federated Training and evaluations with ease, while providing the flexibility to experiment with new algorithms through the low-level interfaces. This flexibility is crucial to the project’s goal of innovating in hate speech detection, a dynamically evolving and sensitive field. TFF’s open-source nature and strong focus on research and experimentation further enhance its suitability, providing a robust and flexible platform that not only satisfies current guidelines, but also offers ample scope for future explorations and improvements to its detection model hate speech.

3.5.1.4 Comparison of TensorFlow Federated with other FL frameworks

TFF stands out in the landscape of FL frameworks due to its unique features and the powerful environment that TensorFlow provides. Compared to other frameworks, such as PySyft, FATE, FedML, and Flower, TFF offers integration with TensorFlow models, which many developers are already familiar with, and supports advanced TensorFlow features such as custom layers and training loops. Below is a comparison table highlighting the advantages of TFF over other FL frameworks:

Table 3.2: Framework Comparison

Feature	TFF	PySyft	FATE	FedML	Flower
Privacy Preserving	Integrated Differential Privacy	Extensions for Differential Privacy	Strong emphasis on security	Multiple privacy approaches supported	Supports secure protocols
Scalability	High, for large-scale deployment	Scalable, with a research focus	High, for enterprise use	High, diverse environments	High, flexible, and scalable
Model Compatibility	TensorFlow and Keras models	PyTorch, TensorFlow, and more	Selective, based on the framework	Broad model support	Broad model support
Communication Protocols	Optimized gRPC	Various, including WebSockets	Custom protocols	Multiple protocols including gRPC	gRPC and RESTful APIs
Execution Environment	Flexible device support	Secure execution environment	Cloud and server-based	Heterogeneous device compatibility	Cross-platform support

3.5.2 Code Editor

The code was created using Visual Studio Code (VSCode), a flexible editor that supports the integration of numerous extensions tailored to the user's coding preferences. Recognized for its minimal RAM consumption, VSCode has been recognized for both its efficiency and ease of use, making it an ideal choice for developers looking for a lightweight yet powerful coding environment.

3.5.3 Python

Python is recognized as an advanced, object-oriented programming language characterized by its dynamic semantics. It is widely regarded as the ideal choice for developing artificial intelligence systems due to its simplicity and readability. Complex topics such as linear algebra and calculus require significant effort to learn, however Python's simple syntax and rapid implementation facilitate immediate validation of concepts by machine learning practitioners [40]. A key factor that favors Python for machine learning projects is its extensive library support. Libraries in programming provide a set of pre-written functions that can be exploited, greatly simplifying the task of coding by reducing the need to develop common functions from scratch. This feature is particularly beneficial in machine learning, where computations involving mathematical optimization, probability, and statistics are routine. Python's rich set of libraries simplifies the execution of complex analyses, allowing data scientists to conduct their research efficiently. The specific libraries applied to this model are described in detail in the following subsections.

3.5.4 Libraries Used

os: This library provides a portable way of using operating system-dependent functions, such as reading or writing files to the system (e.g. creating a directory). It is used to create directories where the script can store output, ensuring that the output of the script is organized and accessible.

time: allows time to be measured, including waiting for a specific duration or measuring the execution time of various blocks of code. It is applied to measure and report the duration of certain operations, such as model training, to help optimize performance.

numpy (np): A library for numerical computations that provides support for tables and high-level mathematical functions for the operation of these data structures. It is used for efficient operations on tables, such as data set shuffling (random reordering of elements), division (splitting into sub-tables), and other arithmetic operations necessary for data pre-processing and manipulation.

pandas (pd): offers data structures (such as DataFrames) and data analysis tools, making it easy to manipulate and analyse tabular data. It is used to load and preprocess data, allowing easy manipulation, filtering and transformation of tabular datasets loaded from CSV files.

matplotlib.pyplot (plt): a graph library used to create static, interactive and animated visualizations in Python. It is used here to graphically display training metrics such as loss and accuracy.

re: Supports regular expression functions, which are powerful tools for searching and manipulating pattern-based strings. It is used for text pre-processing tasks, such as removing URLs or specific patterns from text data, making the data cleaner for NLP tasks.

string: this library contains common string operations and constants. It is used for string manipulation tasks, such as removing punctuation from text, which is a common step in text preprocessing to normalize data.

nltk: It is a library used for working with human language (text) data. It is used for removing stopwords, tokenizing, and lemmatizing text data.

tensorflow (tf): an open-source library used for machine learning that allows developers to create complex ML models with ease. Its use is for model creation, training and evaluation.

tensorflow_federated (tff): is an open source framework for machine learning and other computations on decentralized data. It is used to implement FL, where the model is trained on multiple clients without exchanging their data.

sklearn.model_selection.train_test_split: A helper function from scikit-learn that helps split the dataset into a training set and a test set, ensuring that the model can be trained on a subset of the data and validated on a non-obvious subset to evaluate its performance.

sklearn.metrics: contains functions for calculating various performance metrics such as accuracy, precision, recall, ROC-AUC and F1 score. These metrics are used to evaluate the performance of the machine learning model.

tensorflow.keras.preprocessing.text.Tokenizer: is a library for preprocessing text and for converting text into sequences of integer numbers or vectors where each tokenization can be identified by a unique number. It converts text data into a format that can be fed into neural network models by converting the text into tokens and converting each token into a numeric identifier.

tensorflow.keras.preprocessing.sequence.pad_sequences: used to ensure that all sequences in a list have the same length by padding them with zeros or truncating them. Necessary for preparing text data sequences for neural network training, ensuring that the input tensors in the model have uniform dimensions.

tensorflow.keras.regularizers: offers regularization techniques, such as L1 and L2 regularization, which are strategies to avoid overfitting by penalizing large weights during training. It is applied to the model definition to incorporate regularization directly into the model architecture, helping to improve generalization by reducing overfitting.

emoji: Provides tools for working with emojis, allowing their removal or conversion to text. This can be important for preserving emotion or other linguistic features conveyed by emojis in text data.

3.6 Experimental Evaluation

3.6.1 Experiments

For this thesis we conducted two experiments.

1) Non-IID vs. IID FL Training

This experiment explores the impact of data distribution—specifically IID versus non-IID data on the performance of our federated learning model. The variation in data distribution was controlled by adjusting the parameter “alpha,” which determines the skewness or non-IIDness of the data distribution across clients:

- **IID Setting:** In the IID scenario, the dataset is evenly split among “x” clients, where each client receives an equal portion of the data, “y,” ensuring a balanced and uniform dataset across all clients. This setup aims to simulate a best-case scenario where each client’s data is representative of the overall dataset, minimizing variability in data distribution.
- **Non-IID Setting:** Here, the skewness of data distribution is adjusted using the “alpha” parameter. Lower values of alpha result in higher skewness, meaning the data distribution among clients is uneven. We test two similar non-IID methods [12], quantity skew and distribution-based label imbalance¹. Quantity Skew, refers to the intentional variation in the total amount of data allocated to each client using a Dirichlet distribution. This strategy applies to real-world scenarios where different clients possess varying volumes of data, potentially impacting learning outcomes and model performance. By controlling the parameter alpha of the Dirichlet distribution, we can simulate environments with light to extreme data quantity imbalances among clients, thus studying the robustness of federated learning algorithms under diverse data availability conditions. Distribution-based label imbalance involves distributing the instances of different classes unevenly among clients, also guided by a Dirichlet distribution. This approach helps simulate scenarios where clients have datasets that are not representative of the global class distribution, a common challenge in distributed settings. Below we show the data distribution among 100 clients.

¹<https://github.com/Xtra-Computing/NIID-Bench>

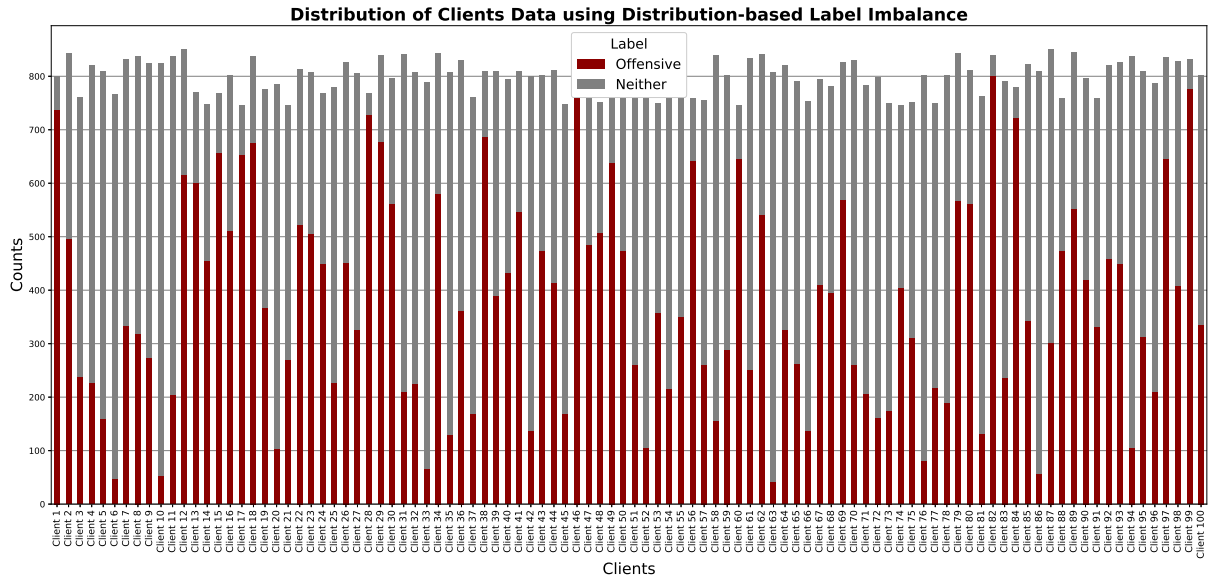


Figure 3.2: Data distribution across 100 clients for Abusive dataset using Distribution-based Label Imbalance

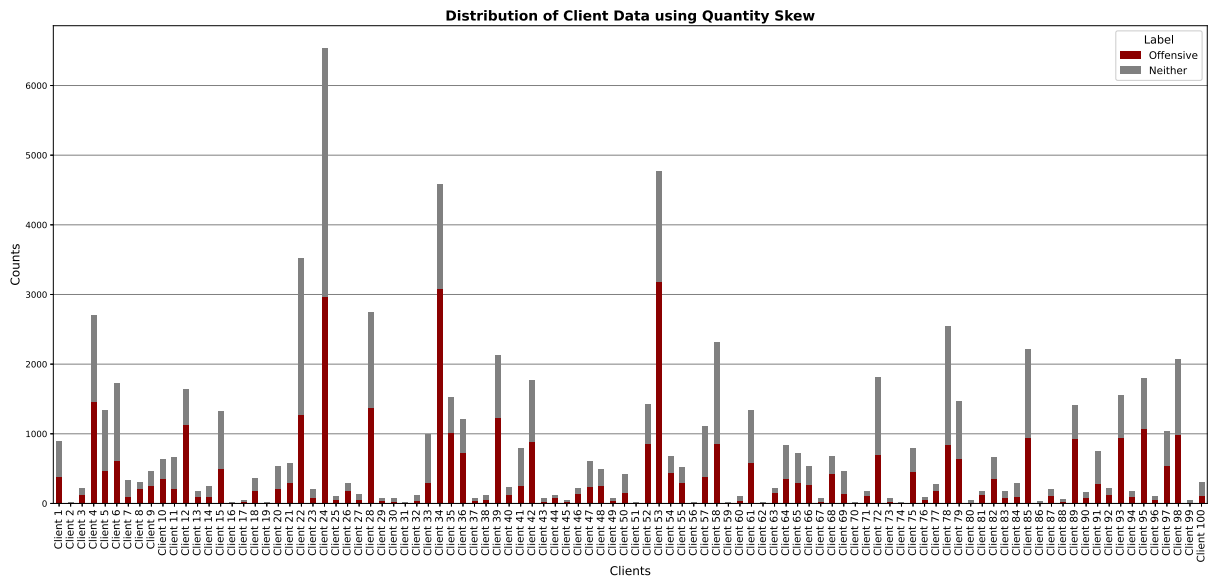


Figure 3.3: Data distribution across 100 clients for Abusive dataset using Quantity Skew

The purpose of this experiment is to compare how well our model perform under ideal versus challenging data conditions compared with other similar models, highlighting the effect of data heterogeneity on model training and performance.

2) Experiment with a “Real” Distributed Dataset (MM-Covid) with 50 and 100 Clients

The second experiment focuses on the application of federated learning to a real-world, distributed dataset concerning COVID-19 misinformation (labeled MM-Covid). In this experiment we wanted to observe how our model performs under real-life scenarios and how the number of clients affects AUC. This dataset involves two setups:

- **50 Clients:** In this setup, the MM-Covid dataset is distributed among 50 clients. This experiment aims to observe the performance and learning dynamics in a moderately large federated network, providing insights into how the model copes with a realistic number of nodes.
- **100 Clients:** Expanding the experiment to 100 clients tests the scalability and efficiency of the federated learning model under a larger and potentially more complex federated environment. This setup helps in understanding how increased client numbers affect model convergence, learning speed, and overall accuracy. Below we show the data distribution among 100 clients.

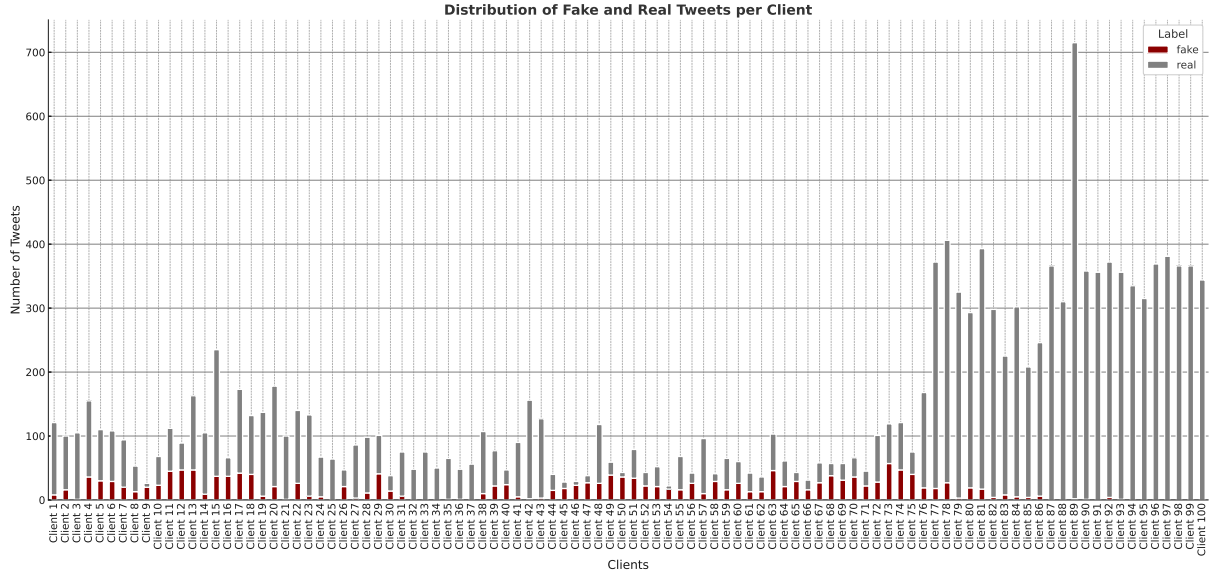


Figure 3.4: Data distribution across 100 clients for MM-Covid dataset

In both setups, each client has a unique dataset. Both of the above experiments were also tested under centralized training with 1 client having the entire dataset. The dataset was then splitted into 80% train and 20% test set for training and evaluation.

3.6.2 Setup

For the setup, we train the text classifier described in Section 3.3, and we compute the weighted classification metrics. We set the parameters (rounds=60, epochs=6, batch size=10, Adams optimizer, learning rate=0.001) after experimenting with different values for tuning (Figure 4.3).

3.6.3 Preparing data for training

For the training, we experimented with 5 different datasets. The following steps show the preparing of data of the Offensive dataset as an example.

Training a text classifier model is divided into 4 steps.

Step 1: Preprocessing Text Data

Initially, the dataset consisted of a large number of tweets, each with varying content and context. The preprocessing stage was critical, removing URLs, emojis, punctuation, and stopwords to reduce the textual material to its essence. Furthermore, lemmatization was used to combine many variants of a word

into a single item, improving the model's capacity to recognize and classify textual patterns effectively. This preprocessing was done with a Python function that used modules like re, string, and nltk. During this procedure, the original data displaying tweets in their raw form was changed into a cleaner, more analysis-friendly format, which was required for the next steps of the machine learning pipeline.

```
Original Data:
                                tweet      label
0  ! ! !   @mayasolovely: As a woman you shouldn ...   neither
1  ! ! ! ! !   @mleew17: boy dats cold...tyga dwn...  offensive
2  ! ! ! ! ! ! !   @UrKindOfBrand Dawg! ! ! !   @...  offensive
3  ! ! ! ! ! ! ! ! !   @C_G_Anderson: @viva_based...  offensive
4  ! ! ! ! ! ! ! ! ! ! ! !   @ShenikaRoberts: T...  offensive
Data shape: (24783, 2)
Data type of tweets: object
```

Figure 3.5: Original Data

```
Data after preprocessing:
                                tweet      label
0  woman complain cleaning house amp man always t...   neither
1  boy dat coldtyga dwn bad cuffin dat hoe st place  offensive
2  dawg ever fuck bitch start cry confused shit      offensive
3  look like tranny                                  offensive
4  shit hear might true might faker bitch told ya    offensive
Data type of tweets: object
```

Figure 3.6: Preprocessed Data

Step 2: Preparing the Dataset

Following preprocessing, each tweet was labeled as 'offensive' or 'neither', converting raw tweets into structured data suitable for supervised learning. This stage is crucial for training the model to accurately differentiate between different categories of content. The dataset was then shuffled to ensure randomization, removing any potential bias caused by the tweets' original sequence. A randomized dataset is essential for developing a robust and unbiased model. Following shuffling, the dataset was partitioned into training and testing sets with an 80-20 split, enabling an organized approach to model training and evaluation. The processed tweets were then tokenized and padded to achieve equal sequence lengths, which met the neural network model's input criteria.

```
Data after labeling:
                                tweet  label
0  woman complain cleaning house amp man always t...      0
1   boy dat coldtyga dwn bad cuffin dat hoe st place      1
2    dawg ever fuck bitch start cry confused shit      1
3                                look like tranny      1
4   shit hear might true might faker bitch told ya      1
Minimum tweet length (in words): 1
Maximum tweet length (in words): 25
Data type of tweets: object
```

Figure 3.7: Labeled Data

```
Data after shuffling:
                                tweet  label
0    bird singing brings thought hot flesh spring m...      1
1    bullet stick niggah damn liver                      1
2  daughter offensive freddy krueger mask got cri...      1
3    many time hoe look picture day                      1
4    u hit list bitch kill yo self                       1
...
24776 karlie jus put bitch nigga better start checki...      1
24777                                put place cause pussy get pussy      1
24778 funny swaggot white boy wanna act like ballin ...      1
24779                                invest curtain bitch      1
24780 puppet like intentionally forever cripple peop...      0
```

Figure 3.8: Shuffled Data

```

First 5 sequences: [[8, 1595, 650, 1, 4031, 5104, 811, 273, 2040], [5105, 137, 1],
[1, 7, 232, 544, 167], [1063, 1243, 77], [193, 43, 218, 100, 497, 302, 227, 101,
1476, 7289, 1]]
First 5 padded sequences: [[ 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0
0 0 0 0 0 0 0 0 8 1595 650 1 4031 5104 811
273 2040]
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 5105
137 1]
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 1 7 232
544 167]
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 1063
1243 77]
[ 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 193 43 218 100 497 302 227 101 1476
7289 1]]

```

Figure 3.9: Tokenized and Padded Data

Step 3: Configuring the Training Pipeline

The training pipeline was setup to take advantage of TensorFlow Federated (TFF), a technology developed for federated learning. This method allowed for distributed model training across numerous clients, which improved privacy and efficiency. A custom model architecture was defined, which included layers such as Embedding, Bidirectional GRU, and Dense, as well as regularization and dropout to reduce overfitting. The training pipeline's setup was critical in allowing the model to effectively learn from decentralized data sources.

Step 4: Training the Model

TensorFlow Federated (TFF) was used to train the model, which was spread across numerous clients to simulate a real-world federated learning scenario. This technique stressed privacy by design while also demonstrating the model's versatility in learning from decentralized datasets. The training used a federated averaging approach, which iteratively improved the model's performance by aggregating updates from participating clients.

4 Results/ Discussion

In this thesis, the train-test split was devised by aggregating tweets from all clients, eliminating duplicates to form a unified dataset, from which 20% was reserved as a global, unseen test set. The test set was then subtracted from each client’s data, ensuring that the training data for each client consisted only of unique tweets not included in the test set. Clients left without data post-separation were excluded from the analysis.

When comparing the results from this thesis to those in the “Privacy–Preserving Online Content Moderation: A Federated Learning Use Case” [3] study, which focused on hate speech datasets, a noticeable variation in performance metrics was observed. The differences in accuracy and F1 scores ranged from 3–20% across both non-IID and centralized training setups. This variation underscores the challenges and potential of federated learning in managing diverse data distributions.

Further comparison with the HypergraphDis model [44], which specifically addresses MM-Covid data related to misinformation, reveals additional insights. The HypergraphDis model demonstrates slightly higher accuracy and F1 scores, showing improvements in the range of 7–9% for both 50 and 100 clients. This indicates that the HypergraphDis model might be better optimized for handling misinformation content within the federated learning framework.

When comparing the results for different alpha values (Table 4.1), we observed that a higher alpha results in a more uniform distribution and thus performance. This contrasts with the IID method, where the performance was originally expected high. Additionally, all performances in the centralized approach surpassed those in scenarios with 50 and 100 clients (Table 4.2).

The Figure 4.2 demonstrates that increasing the number of clients in a federated learning setup on the MM-Covid dataset leads to a consistent improvement in the AUC. This is likely due to the aggregation of more diverse data from multiple sources. This diversity helps the model to generalize better across different scenarios, reducing overfitting and improving its predictive accuracy on varied data.

Overall, our model demonstrated robust performance in real-life non-IID scenarios, indicating its effectiveness under varied conditions.” This enhancement highlights the model’s adaptability and reliability across different situations.

Table 4.1: F1 score (standard deviation noted) for hate speech datasets for different alpha variations and IID for 100 clients using quantity skew and distribution-based label imbalance

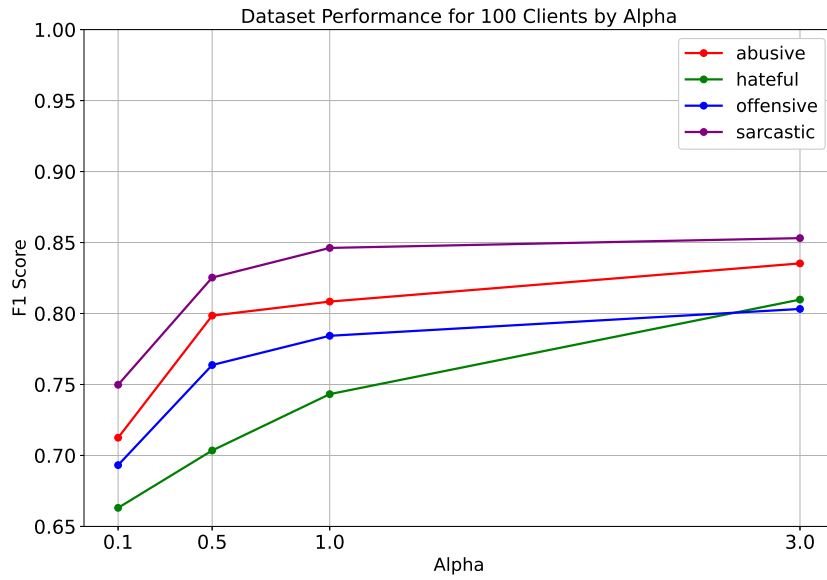
Dataset	Alpha	F1 Score (weighted) Quantity skew	F1 Score (weighted) Distribution-based label imbalance	IID
Abusive	0.1	0.72 ± 0.02	0.68 ± 0.02	0.89 ± 0.01
	0.5	0.79 ± 0.02	0.74 ± 0.02	
	1.0	0.81 ± 0.02	0.78 ± 0.02	
	3.0	0.84 ± 0.02	0.84 ± 0.02	
Hateful	0.1	0.67 ± 0.01	0.63 ± 0.01	0.87 ± 0.01
	0.5	0.70 ± 0.01	0.68 ± 0.01	
	1.0	0.74 ± 0.01	0.74 ± 0.01	
	3.0	0.81 ± 0.02	0.80 ± 0.01	
Sarcastic	0.1	0.75 ± 0.01	0.70 ± 0.01	0.90 ± 0.01
	0.5	0.82 ± 0.02	0.77 ± 0.01	
	1.0	0.84 ± 0.02	0.83 ± 0.01	
	3.0	0.85 ± 0.01	0.86 ± 0.01	
Offensive	0.1	0.68 ± 0.01	0.66 ± 0.01	0.91 ± 0.01
	0.5	0.76 ± 0.02	0.71 ± 0.01	
	1.0	0.78 ± 0.01	0.77 ± 0.01	
	3.0	0.81 ± 0.01	0.83 ± 0.01	

Table 4.2: The performance metrics for the hate speech datasets were evaluated under a **non-iid condition** using the **quantity skew method**, as previously discussed, with an **alpha=0.5**.

Abusive Content: non-IID clients construction				
Dataset	# Clients	Accuracy (non-IID)	F1 Score (weighted) (non-IID)	F1 Score (weighted) (Centralized)
Abusive	50	0.73 ± 0.01	0.73 ± 0.01	0.82 ± 0.02
	100	0.77 ± 0.01	0.79 ± 0.02	
Hateful	50	0.71 ± 0.01	0.73 ± 0.01	0.77 ± 0.01
	100	0.72 ± 0.02	0.70 ± 0.01	
Sarcastic	50	0.86 ± 0.02	0.84 ± 0.01	0.88 ± 0.01
	100	0.87 ± 0.01	0.82 ± 0.02	
Offensive	50	0.85 ± 0.02	0.82 ± 0.02	0.88 ± 0.02
	100	0.82 ± 0.02	0.76 ± 0.02	

Table 4.3: The MM-Covid dataset went testing under actual non-iid conditions. This dataset was subjected to experiments conducted over 6 epochs and 60 rounds. The results presented are average values calculated from five repetitions, with the standard deviation noted. This table focuses on the second experiment.

Disinformation: The clients are a superset of Twitter users based on social network & METIS graph partitioning				
Dataset	# Clients	Accuracy (non-IID)	F1 Score (weighted) non-IID	F1 Score (weighted) Centralized
MM-Covid	50	0.78 ± 0.01	0.77 ± 0.03	0.84 ± 0.02
	100	0.79 ± 0.02	0.79 ± 0.01	



(a) Quantity skew method



(b) Distribution-based label imbalance method

Figure 4.1: Comparison of F1 scores for different alpha variations and different non-iid methods 30

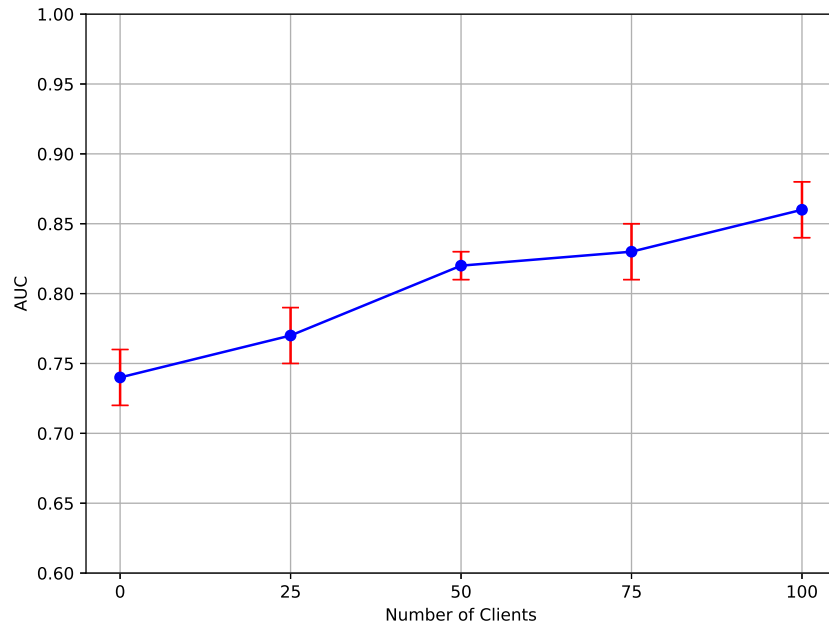


Figure 4.2: Comparison between AUC and number of clients for MM-Covid dataset over 5 repetitions, 60 rounds and 6 epochs.

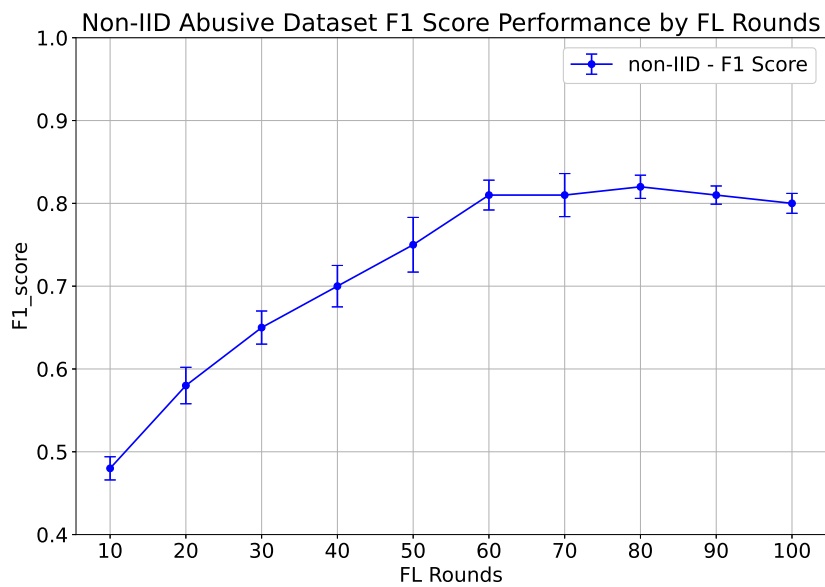


Figure 4.3: Model convergence for Abusive dataset over 5 repetitions for 100 clients. In this graph we show the reason we chose the number of training rounds discussed in Section 3.6.2

5 Future Work

The research conducted in this thesis lays the groundwork for several promising future directions. One immediate addition would be to investigate the federated learning model’s scalability on larger, more diverse datasets. Further work could look into the effect of such scaling on the model’s accuracy and communication costs, potentially leading to the development of more effective aggregation algorithms or compression approaches to reduce bandwidth usage.

Another potential avenue is the incorporation of extra linguistic elements into the model. The current model could be improved by including semantic analysis techniques that account for sarcasm, idioms, and regional dialects, which are frequently problematic for sentiment analysis algorithms. Furthermore, diving into multilingual and cross-lingual model training could significantly broaden the research’s usefulness, allowing for global deployment on platforms with various user populations.

The topic of privacy in federated learning deserves additional investigation. Advanced cryptographic techniques, such as secure multi-party computation or homomorphic encryption, could improve the privacy-preserving properties of the federated learning process. Addressing concerns about user anonymity and data security in accordance with growing rules and ethical standards would be timely and effective.

Experimenting with different neural network topologies, such as transformer models or advanced RNN variations, may provide further insight into model performance. Furthermore, the federated learning framework might be used to examine the idea of transfer learning, which involves fine-tuning a model pre-trained on a broad corpus using domain-specific data. This method could reduce the cold start problem in federated environments, resulting in more resilient models with less training data.

Finally, user experience studies would be extremely beneficial in understanding the impact of such models when deployed in real-world circumstances. Gathering user feedback on the model’s classifications and adding it into model retraining could contribute to a more user-centered approach to automated content management. Furthermore, the creation of tools to explain model decisions could promote confidence and transparency, resulting in greater adoption.

Through these many extensions, the work undertaken in this thesis may continue to expand, making substantial contributions to the fields of NLP, machine learning, and online social interaction. The intersection of these disciplines constitutes a fertile field of study, ready for future academic research and practical application.

BIBLIOGRAPHY

- [1] G. A. Blog. (2017) Federated learning: Collaborative machine learning without centralized training data. [Online]. Available: <https://ai.googleblog.com/2017/04/federated-learning-collaborative.html?m=1>
- [2] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, and K. Seth, “Practical secure aggregation for privacy-preserving machine learning,” in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 2017. [Online]. Available: <https://dl.acm.org/doi/pdf/10.1145/3133956.3133982>
- [3] P. Leonidou, N. Kourtellis, N. Salamanos, and M. Sirivianos, “Privacy-preserving online content moderation: A federated learning use case,” in *Companion Proceedings of the ACM Web Conference 2023*, ser. WWW ’23 Companion. New York, NY, USA: Association for Computing Machinery, 2023, p. 280–289. [Online]. Available: <https://doi.org/10.1145/3543873.3587604>
- [4] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas. (2016) Communication-efficient learning of deep networks from decentralized data. [Online]. Available: <https://arxiv.org/pdf/1602.05629.pdf>
- [5] K. Bonawitz, H. Eichner, W. Grieskamp, D. Huba, A. Ingerman, V. Ivanov, C. Kiddon, J. Konečný, S. Mazzocchi, H. B. McMahan, T. Van Overveldt, D. Petrou, D. Ramage, and J. Roselander. (2019) Towards federated learning at scale: System design. [Online]. Available: <https://arxiv.org/pdf/1902.01046.pdf>
- [6] A. Founta, C. Djouvas, D. Chatzakou, I. Leontiadis, J. Blackburn, G. Stringhini, A. Vakali, M. Sirivianos, and N. Kourtellis, “Large scale crowdsourcing and characterization of twitter abusive behavior,” in *Proceedings of the International AAAI Conference on Web and Social Media*, vol. 12, no. 1, 2018.
- [7] A. M. Founta, D. Chatzakou, N. Kourtellis, J. Blackburn, A. Vakali, and I. Leontiadis, “A unified deep learning architecture for abuse detection,” in *Proceedings of the 10th ACM Conference on Web Science*, ser. WebSci ’19. New York, NY, USA: ACM, 2019, pp. 105–114.
- [8] T. Davidson, D. Warmley, M. W. Macy, and I. Weber, “Automated hate speech detection and the problem of offensive language,” in *Proceedings of the Eleventh International Conference on Web and Social Media, ICWSM 2017*. Montreal, Québec, Canada: AAAI Press, May 2017, pp. 512–515.
- [9] Z. Waseem and D. Hovy, “Hateful symbols or hateful people? predictive features for hate speech detection on twitter,” in *Proceedings of the NAACL Student Research Workshop*. San Diego, California: Association for Computational Linguistics, 2016, pp. 88–93.
- [10] A. Rajadesingan, R. Zafarani, and H. Liu, “Sarcasm detection on twitter: A behavioral modeling approach,” in *Proceedings of the Eighth ACM International Conference on Web Search and Data Mining*. New York, NY, USA: ACM, 2015, pp. 97–106.
- [11] Y. Li, B. Jiang, K. Shu, and H. Liu, “Mm-covid: A multilingual and multimodal data repository for combating covid-19 disinformation,” 2020.

- [12] Q. Li, Y. Diao, Q. Chen, and B. He, “Federated learning on non-iid data silos: An experimental study,” *arXiv preprint arXiv:2102.02079*, 2021. [Online]. Available: <https://arxiv.org/pdf/2102.02079.pdf>
- [13] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith, “Federated learning: Challenges, methods, and future directions,” *IEEE Signal Processing Magazine*, vol. 37, no. 3, pp. 50–60, 2020. [Online]. Available: <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=9084352>
- [14] M. Chen, B. Abdelkader, C. Fung, and C. Yau. (2019) Federated learning of out-of-vocabulary words. [Online]. Available: <https://arxiv.org/pdf/1903.10635.pdf>
- [15] H. B. McMahan and D. Ramage. (2017) Federated optimization: Distributed machine learning for on-device intelligence. [Online]. Available: <https://arxiv.org/pdf/1610.02527.pdf>
- [16] J. Konečný, H. B. McMahan, F. X. Yu, P. Richtárik, A. T. Suresh, and D. Bacon. (2016) Federated learning: Strategies for improving communication efficiency. [Online]. Available: <https://arxiv.org/pdf/1610.05492.pdf>
- [17] S. J. Reddi, Z. Charles, M. Zaheer, Z. Garrett, K. Rush, J. Konečný, S. Kumar, and H. B. McMahan. (2020) Adaptive federated optimization. [Online]. Available: <https://arxiv.org/pdf/2003.00295.pdf>
- [18] M. Abadi and H. B. McMahan. (2017) Differentially private federated learning: A client level perspective. [Online]. Available: <https://arxiv.org/pdf/1712.07557.pdf>
- [19] L. Lyu, H. Yu, and Q. Yang, “Privacy and robustness in federated learning: Attacks and defenses,” *IEEE Communications Surveys Tutorials*, vol. 23, no. 1, pp. 746–769, 2021. [Online]. Available: <https://ieeexplore.ieee.org/document/9945997>
- [20] H. Jin, Y. Miao, L. Liu, J. Song, and Y. Wang. (2020) Local and central differential privacy for robustness and privacy in federated learning. [Online]. Available: <https://arxiv.org/abs/2009.03561>
- [21] S. Wang, T. Tuor, T. Salonidis, K. K. Leung, C. Makaya, T. He, and K. Chan, “Fededge: Accelerating edge-assisted federated learning,” in *Proceedings of the ACM SIGCOMM 2022 Conference*, 2022. [Online]. Available: <https://doi.org/10.1145/3543507.3583264>
- [22] D. M. West, “What is artificial intelligence?” www.brookings.edu/research/what-is-artificial-intelligence/, Oct. 2018, brookings, Brookings, 4 Oct. 2018.
- [23] expert.ai, “What is machine learning? a definition,” www.expert.ai/blog/machine-learning-definition/#:~:text=Machine%20learning%20is%20an%20application, May 2020, expert System, 5 May 2020.
- [24] V. Advani, “What is artificial intelligence? how does ai work and future of it,” www.mygreatlearning.com/blog/what-is-artificial-intelligence/, Apr. 2020, greatLearning, 23 Apr. 2020.
- [25] B. Dickson, “What is deep learning?” bdtechtalks.com/2019/02/15/what-is-deep-learning-neural-networks/, Feb. 2019, techTalks, 15 Feb. 2019.
- [26] D. Bhavsar, “Dispelling myths: Deep learning vs. machine learning | blog,” www.merkleinc.com/blog/dispelling-myths-deep-learning-vs-machine-learning, May 2020, merkle, 6 May 2020.
- [27] P. Kairouz *et al.*, “Title of the paper,” *arXiv:1912.04977*, 2019. [Online]. Available: <https://arxiv.org/abs/1912.04977>

- [28] K. Bonawitz, H. Eichner, W. Grieskamp, D. Huba, A. Ingerman, V. Ivanov *et al.*, “Towards federated learning at scale: System design,” *arXiv preprint arXiv:1902.01046*, 2019.
- [29] Q. Yang, Y. Liu, T. Chen, and Y. Tong, “Federated machine learning: Concept and applications,” *ACM Trans. Intell. Syst. Technol.*, vol. 10, no. 2, pp. 12:1–12:19, 2019.
- [30] D. Jurafsky and J. H. Martin, *Speech and Language Processing*, 3rd ed. Stanford University, 2019. [Online]. Available: <https://web.stanford.edu/~jurafsky/slp3/ed3book.pdf>
- [31] Y. Bengio, R. Ducharme, P. Vincent, and C. Jauvin, “A neural probabilistic language model,” *Journal of Machine Learning Research*, vol. 3, pp. 1137–1155, 2003. [Online]. Available: <https://www.jmlr.org/papers/volume3/bengio03a/bengio03a.pdf>
- [32] C. D. Manning and H. Schütze, *Foundations of Statistical Natural Language Processing*. MIT Press, 1999. [Online]. Available: <https://www.scirp.org/reference/referencespapers?referenceid=1956785>
- [33] K. Cho *et al.*, “Learning phrase representations using rnn encoder-decoder for statistical machine translation,” *arXiv preprint arXiv:1406.1078*, 2014. [Online]. Available: <https://arxiv.org/abs/1406.1078>
- [34] J. Devlin *et al.*, “Bert: Pre-training of deep bidirectional transformers for language understanding,” *arXiv preprint arXiv:1810.04805*, 2018. [Online]. Available: <https://arxiv.org/abs/1810.04805>
- [35] K. Papineni *et al.*, “Bleu: a method for automatic evaluation of machine translation,” in *Proceedings of the 40th Annual Meeting on Association for Computational Linguistics*, 2002, pp. 311–318. [Online]. Available: <https://aclanthology.org/P02-1040/>
- [36] N. Kholod *et al.*, “Open-source federated learning frameworks for iot: A comparative review and analysis,” ResearchGate, 2020. [Online]. Available: https://www.researchgate.net/publication/348067413_Open-Source_Federated_Learning_Frameworks_for_IoT_A_Comparative_Review_and_Analysis
- [37] C. Jeeva, “Introduction to recurrent neural network (rnn),” Scaler Topics, 2020, accessed: 2024-03-17. [Online]. Available: <https://www.scaler.com/topics/deep-learning/rnn/>
- [38] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016. [Online]. Available: <http://www.deeplearningbook.org/>
- [39] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, “Empirical evaluation of gated recurrent neural networks on sequence modeling,” *arXiv preprint arXiv:1412.3555*, 2014. [Online]. Available: <https://arxiv.org/abs/1412.3555>
- [40] A. Brothers, “Why is python so popular in machine learning and ai?” ASPER BROTHERS, Oct. 2019. [Online]. Available: <https://asperbrothers.com/blog/why-python-for-machine-learning/#:~:text=Python%20is%20undoubtedly%20the%20best>
- [41] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014.

- [42] N. Srivastava and R. Salakhutdinov, “Modeling documents with deep boltzmann machines,” *arXiv preprint arXiv:1309.6865*, 2013. [Online]. Available: <https://arxiv.org/abs/1309.6865>
- [43] X. Sun, B. Wang, S. Liu, and T. Liu, “Lmc-smca: A new active learning method in asr,” *IEEE Access*, 2021.
- [44] N. Salamanos, P. Leonidou, N. Laoutaris, M. Sirivianos, M. Aspri, and M. Paraschiv, “Hypergraphdis: Leveraging hypergraphs for contextual and social-based disinformation detection,” in *Name of the Conference or Workshop*. Conference Location: Publisher Name, 2022, pp. Start Page–End Page.
- [45] t. **Salamanos**, P. Leonidou, N. Laoutaris, M. Sirivianos, M. Aspri, and M. Paraschiv, “A unified graph-based approach to disinformation detection using contextual and semantic relations,” *18th International AAAI Conference on Web and Social Media (ICWSM 2024)* – ***In press***, 2024, in press.